

性能

AI初始版本

```
1  #include <stdbool.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  bool isIPv4Address(const char* addr);
6  bool isIPv6Address(const char* addr);
7
8  bool isDomainName(const char* addr) {
9      // 首先检查是否是IPv4或IPv6地址
10     if (isIPv4Address(addr) || isIPv6Address(addr)) {
11         return false;
12     }
13
14     int len = strlen(addr);
15     if (len < 1 || len > 253) { // 域名最大长度253字符
16         return false;
17     }
18
19     // 检查是否包含非法字符
20     for (int i = 0; i < len; i++) {
21         char c = addr[i];
22         // 允许的字符: 字母、数字、连字符、点号
23         if (!(isalnum(c) || c == '-' || c == '.')) {
24             return false;
25         }
26     }
27
28     // 不能以点号或连字符开头或结尾
29     if (addr[0] == '.' || addr[0] == '-' ||
30         addr[len - 1] == '.' || addr[len - 1] == '-') {
31         return false;
32     }
33
34     // 检查标签 (label) 规则
35     int label_length = 0;
36     bool has_dot = false;
37
38     for (int i = 0; i < len; i++) {
39         if (addr[i] == '.') {
40             has_dot = true;
41             // 标签不能以连字符开头或结尾
42             if (i > 0 && (addr[i - 1] == '-' || (i + 1 < len && addr[i + 1] == '-'))))
43                 return false;
44         }
45     }
```

```

45         // 连续的点号
46         if (i > 0 && addr[i - 1] == '.') {
47             return false;
48         }
49         // 标签长度检查 (1-63字符)
50         if (label_length < 1 || label_length > 63) {
51             return false;
52         }
53         label_length = 0;
54     }
55     else {
56         label_length++;
57         // 单个标签不能超过63字符
58         if (label_length > 63) {
59             return false;
60         }
61     }
62 }
63
64 // 最后一个标签的长度检查
65 if (label_length < 1 || label_length > 63) {
66     return false;
67 }
68
69 // 顶级域名 (最后一个标签) 应该是字母
70 const char* last_dot = strrchr(addr, '.');
71 const char* tld = last_dot ? last_dot + 1 : addr;
72 int tld_len = strlen(tld);
73
74 // 顶级域名应该至少包含一个字母
75 bool has_alpha = false;
76 for (int i = 0; i < tld_len; i++) {
77     if (isalpha(tld[i])) {
78         has_alpha = true;
79         break;
80     }
81 }
82
83 if (!has_alpha) {
84     return false;
85 }
86
87 return true;
88 }
89
90 // 简化的IPv4地址检查函数 (供参考)
91 bool isIPv4Address(const char* addr) {
92     int parts = 0;
93     int num = 0;
94     int digits = 0;
95
96     for (int i = 0; addr[i] != '\0'; i++) {
97         if (addr[i] == '.') {

```

```

98         if (digits == 0 || num > 255) return false;
99         parts++;
100         num = 0;
101         digits = 0;
102     }
103     else if (isdigit(addr[i])) {
104         if (digits > 0 && num == 0) return false; // 前导零
105         num = num * 10 + (addr[i] - '0');
106         digits++;
107     }
108     else {
109         return false;
110     }
111 }
112
113 if (digits == 0 || num > 255) return false;
114 parts++;
115
116 return parts == 4;
117 }
118
119 // 简化的IPv6地址检查函数 (供参考)
120 bool isIPv6Address(const char* addr) {
121     int colons = 0;
122     int digits = 0;
123     bool has_double_colon = false;
124
125     for (int i = 0; addr[i] != '\0'; i++) {
126         if (addr[i] == ':') {
127             colons++;
128             if (i > 0 && addr[i - 1] == ':') {
129                 if (has_double_colon) return false; // 多个 "::"
130                 has_double_colon = true;
131             }
132             digits = 0;
133         }
134         else if (isdigit(addr[i])) {
135             digits++;
136             if (digits > 4) return false; // 每个段最多4个十六进制数字
137         }
138         else {
139             return false;
140         }
141     }
142
143     return (colons >= 2 && colons <= 7);
144 }

```

测试方法

1. 编写代码随机生成大量域名，计算其平均耗时
2. 随机生成大量域名进行压力测试

3. 边界条件测试

测试结果

```
1  域名验证函数性能测试
2  =====
3  === 性能测试开始 ===
4  测试用例：合法域名          | 平均时间：1.000 微秒 | 总耗时：0.010 秒
5  测试用例：长域名            | 平均时间：3.600 微秒 | 总耗时：0.036 秒
6  测试用例：IPv4地址          | 平均时间：0.400 微秒 | 总耗时：0.004 秒
7  测试用例：IPv6地址          | 平均时间：0.500 微秒 | 总耗时：0.005 秒
8  测试用例：非法字符          | 平均时间：0.200 微秒 | 总耗时：0.002 秒
9  测试用例：点号开头          | 平均时间：0.200 微秒 | 总耗时：0.002 秒
10 测试用例：点号结尾           | 平均时间：0.300 微秒 | 总耗时：0.003 秒
11 测试用例：连字符开头         | 平均时间：0.300 微秒 | 总耗时：0.003 秒
12 测试用例：长标签             | 平均时间：1.100 微秒 | 总耗时：0.011 秒
13
14  === 压力测试开始 ===
15  压力测试结果：
16  总测试数：100000
17  有效域名数：44449
18  总耗时：0.553 秒
19  处理速度：180832 域名/秒
20  平均时间：5.530 微秒/域名
21
22  === 边界条件测试 ===
23  边界测试：空字符串          | 平均时间：0.000 微秒
24  边界测试：最小域名          | 平均时间：0.100 微秒
25  边界测试：最大域名          | 平均时间：3.800 微秒
26  边界测试：超长域名          | 平均时间：3.600 微秒
```

修改要求

- 1. 考虑在检查非法字符时提前返回
- 2. 优化标签长度检查逻辑
- 3. 减少不必要的字符串遍历
- 4. 考虑使用查找表加速字符验证

AI优化版本

```
1  #include <stdbool.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  // 声明IP地址检查函数
6  bool isIPv4Address(const char* addr);
7  bool isIPv6Address(const char* addr);
8
9  // 字符验证查找表（静态常量，避免重复初始化）
10 static const unsigned char VALID_DOMAIN_CHARS[256] = {
```

```

11     ['0'] = 1, ['1'] = 1, ['2'] = 1, ['3'] = 1, ['4'] = 1, ['5'] = 1, ['6'] = 1,
    ['7'] = 1, ['8'] = 1, ['9'] = 1,
12     ['A'] = 1, ['B'] = 1, ['C'] = 1, ['D'] = 1, ['E'] = 1, ['F'] = 1, ['G'] = 1,
    ['H'] = 1, ['I'] = 1, ['J'] = 1,
13     ['K'] = 1, ['L'] = 1, ['M'] = 1, ['N'] = 1, ['O'] = 1, ['P'] = 1, ['Q'] = 1,
    ['R'] = 1, ['S'] = 1, ['T'] = 1,
14     ['U'] = 1, ['V'] = 1, ['W'] = 1, ['X'] = 1, ['Y'] = 1, ['Z'] = 1,
15     ['a'] = 1, ['b'] = 1, ['c'] = 1, ['d'] = 1, ['e'] = 1, ['f'] = 1, ['g'] = 1,
    ['h'] = 1, ['i'] = 1, ['j'] = 1,
16     ['k'] = 1, ['l'] = 1, ['m'] = 1, ['n'] = 1, ['o'] = 1, ['p'] = 1, ['q'] = 1,
    ['r'] = 1, ['s'] = 1, ['t'] = 1,
17     ['u'] = 1, ['v'] = 1, ['w'] = 1, ['x'] = 1, ['y'] = 1, ['z'] = 1,
18     ['-'] = 1, ['.'] = 1
19 };
20
21 bool isDomainName(const char* addr) {
22     // 快速长度检查
23     int len = strlen(addr);
24     if (len < 1 || len > 253) {
25         return false;
26     }
27
28     // 排除IP地址
29     if (isIPv4Address(addr) || isIPv6Address(addr)) {
30         return false;
31     }
32
33     // 边界字符检查
34     unsigned char first_char = (unsigned char)addr[0];
35     unsigned char last_char = (unsigned char)addr[len-1];
36
37     if (first_char == '.' || first_char == '-' ||
38         last_char == '.' || last_char == '-') {
39         return false;
40     }
41
42     // 单次遍历完成所有检查
43     int label_length = 0;
44     bool has_alpha_in_tld = false;
45     bool has_dot = false;
46     char prev_char = 0;
47
48     for (int i = 0; i < len; i++) {
49         unsigned char c = (unsigned char)addr[i];
50
51         // 使用查找表检查非法字符 (最快方式)
52         if (!VALID_DOMAIN_CHARS[c]) {
53             return false;
54         }
55
56         if (c == '.') {
57             has_dot = true;
58

```

```

59         // 检查标签长度
60         if (label_length < 1 || label_length > 63) {
61             return false;
62         }
63
64         // 检查连续点号或点号位置
65         if (prev_char == '.' || (i > 0 && prev_char == '-') ||
66             (i + 1 < len && addr[i + 1] == '-')) {
67             return false;
68         }
69
70         label_length = 0; // 重置标签长度
71         has_alpha_in_tld = false; // 重置TLD字母标志
72     } else {
73         label_length++;
74
75         // 检查单个标签长度 (提前返回)
76         if (label_length > 63) {
77             return false;
78         }
79
80         // 记录TLD中是否有字母
81         if (!has_dot && isalpha(c)) {
82             has_alpha_in_tld = true;
83         }
84     }
85
86     prev_char = c;
87 }
88
89 // 检查最后一个标签长度
90 if (label_length < 1 || label_length > 63) {
91     return false;
92 }
93
94 // 检查顶级域名是否包含字母
95 if (!has_alpha_in_tld) {
96     // 需要验证TLD是否包含字母
97     const char* last_segment = addr;
98     if (has_dot) {
99         const char* last_dot = strrchr(addr, '.');
100         last_segment = last_dot + 1;
101     }
102
103     bool has_alpha = false;
104     for (int i = 0; last_segment[i] != '\0'; i++) {
105         if (isalpha((unsigned char)last_segment[i])) {
106             has_alpha = true;
107             break;
108         }
109     }
110     if (!has_alpha) {
111         return false;

```

```

112     }
113 }
114
115     return true;
116 }
117
118 // 优化的IPv4检查函数
119 bool isIPv4Address(const char* addr) {
120     int parts = 0, num = 0, digits = 0;
121
122     for (int i = 0; addr[i] != '\0'; i++) {
123         if (addr[i] == '.') {
124             if (digits == 0 || num > 255) return false;
125             parts++;
126             num = digits = 0;
127         } else if (isdigit((unsigned char)addr[i])) {
128             if (digits > 0 && num == 0) return false; // 前导零
129             num = num * 10 + (addr[i] - '0');
130             digits++;
131             if (digits > 3) return false;
132         } else {
133             return false;
134         }
135     }
136
137     return (digits > 0 && num <= 255 && parts == 3);
138 }
139
140 // 优化的IPv6检查函数
141 bool isIPv6Address(const char* addr) {
142     int colons = 0, digits = 0;
143     bool has_double_colon = false;
144
145     for (int i = 0; addr[i] != '\0'; i++) {
146         if (addr[i] == ':') {
147             colons++;
148             if (i > 0 && addr[i-1] == ':') {
149                 if (has_double_colon) return false;
150                 has_double_colon = true;
151             }
152             digits = 0;
153         } else if (isdigit((unsigned char)addr[i])) {
154             if (++digits > 4) return false;
155         } else {
156             return false;
157         }
158     }
159
160     return (colons >= 2 && colons <= 7);
161 }

```

优化后性能测试

```
1  域名验证函数性能测试
2  =====
3  === 性能测试开始 ===
4  测试用例：合法域名          | 平均时间：0.100 微秒 | 总耗时：0.001 秒
5  测试用例：长域名            | 平均时间：0.400 微秒 | 总耗时：0.004 秒
6  测试用例：IPv4地址          | 平均时间：0.100 微秒 | 总耗时：0.001 秒
7  测试用例：IPv6地址          | 平均时间：0.500 微秒 | 总耗时：0.005 秒
8  测试用例：非法字符          | 平均时间：0.100 微秒 | 总耗时：0.001 秒
9  测试用例：点号开头          | 平均时间：0.100 微秒 | 总耗时：0.001 秒
10 测试用例：点号结尾           | 平均时间：0.000 微秒 | 总耗时：0.000 秒
11 测试用例：连字符开头         | 平均时间：0.100 微秒 | 总耗时：0.001 秒
12 测试用例：长标签             | 平均时间：1.000 微秒 | 总耗时：0.010 秒
13
14  === 压力测试开始 ===
15  压力测试结果：
16  总测试数：100000
17  有效域名数：44299
18  总耗时：0.399 秒
19  处理速度：250627 域名/秒
20  平均时间：3.990 微秒/域名
21
22  === 边界条件测试 ===
23  边界测试：空字符串          | 平均时间：0.000 微秒
24  边界测试：最小域名          | 平均时间：0.100 微秒
25  边界测试：最大域名          | 平均时间：1.200 微秒
26  边界测试：超长域名          | 平均时间：1.200 微秒
```